SYSC3110 – Milestone 3 3.a UML Class Diagrams – Changes From Milestone 2

This section explains how the UML and data structures from the Milestone 2 diagram were changed to support the Milestone 3 UNO Flip implementation and AI player.

1. Card, CardColor, CardType

In the Milestone 2 UML:
- Card had a single colour (color : CardColor) and a single type (type : CardType).
- CardColor only contained the light-side colours: RED, BLUE, YELLOW, GREEN, WILD.
- CardType only contained the light-side values (ONE–NINE, REVERSE, FLIP, DRAW_ONE, SKIP, WILD, WILD_DRAW_TWO).
- There was no notion of a dark side in the enums or in the Card attributes.

In the Milestone 3 code:
- Card now has two sets of attributes:
• lightColor : CardColor, lightType : CardType
• darkColor : CardColor, darkType : CardType
- Card also stores two images: lightImage and darkImage, one for each side.
- CardColor and CardType have been extended with dark-side values (ORANGE, PINK, PURPLE, TEAL and SKIP_EVERYONE, DRAW_FIVE, WILD_DRAW_COLOR), and each enum constant now carries a boolean isLight attribute.
- Card contains a static Map called Point_System that maps card types to point values and is used by getPoint().

Reason:
- UNO Flip requires every physical card to have a light and dark side. Instead of creating two separate classes, a single Card object now represents both sides and chooses which side is active based on UNOModel.isDarkSide.
- The HashMap centralizes scoring rules and allows O(1) lookup of card points without long switch statements.
- The isLight flag on CardColor and CardType lets PileSystem.createDrawPile() generate only light-side cards for the starting deck and helps map between light and dark equivalents.

2. PileSystem

In the Milestone 2 UML:
- PileSystem already had drawPile : Stack and discardPile : Stack, plus methods such as drawCard(), addToDiscardPile() and peekTopDiscard().

In the Milestone 3 code:
- The same two stacks are used, but the creation logic in createDrawPile() now uses the extended CardColor and CardType enums and their isLightSide() methods to skip dark-side values when building the initial draw pile.
- A restoreDrawPile() method reshuffles cards from discardPile back into drawPile when the deck is empty, keeping the top discard card in place.

Reason:
- Stack remains the right data structure for both piles, because we always interact with the top card (push/pop/peek in O(1) time).
- The main change is that PileSystem now understands the light/dark distinction introduced for UNO Flip.

3. Player and ComputerPlayer

In the Milestone 2 UML:
- Only a Player class was present, with a hand : List (ArrayList) and score fields.

In the Milestone 3 design:
- Player still holds its hand in a List implemented by an ArrayList.
- A new subclass ComputerPlayer extends Player (this may not appear in the old M2 UML at all).
- UNOModel uses instanceof to detect when the current player is a ComputerPlayer and apply AI logic.

Reason:
- The ArrayList hand is reused because it already supports dynamic hand sizes with O(1) index access and simple iteration.
- Introducing ComputerPlayer as a subclass keeps the data structure for hands identical for humans and AI, while clearly separating AI behaviour from human input handling.

4. UNOModel and Observer Support

In the Milestone 2 UML:
- UNOModel already contained players : ArrayList and methods to interact with the model.
- The dependency on IView was present but simpler; there was no UNOEvent class in the diagram and the update() method's parameter was not modelled in detail.

In the Milestone 3 code:
- UNOModel still uses players : ArrayList, but now also maintains extra state to support Flip-specific cards (flags for drawOne, drawFive, wildDrawColor, skipEveryone, and a static isDarkSide boolean).
- viewersList : List is used to register multiple views.
- A new class UNOEvent bundles together Status, playedCard, lastDrawnCard, currentPlayer, nextPlayer,
topCard, wildColor, players and drawCounter. This event object is sent to each IView in notifyViewers().

Reason:
- Keeping players in an ArrayList is still the best fit for index-based turn rotation and score calculation.
- Introducing UNOEvent cleans up the observer interface: instead of the view pulling many pieces of data from UNOModel, the model pushes a single snapshot object. This matches the Observer pattern
more closely and makes the UML relationships between UNOModel, UNOEvent and IView clearer.

5. UNOView and TopPanel

In the Milestone 2 UML:
- UNOView extended JFrame and implemented IView with an update() method, but the event details and
some UI helper structures were not fully modelled.
- TopPanel showed labels for the top card and current player.

In the Milestone 3 code:
- UNOView.update(UNOEvent event) now switches on Status and uses data from UNOEvent (wildColor,
computerWild, players list, etc.) to drive the UI.
- UNOView stores playerScoreLabels : ArrayList to keep the scoreboard aligned with the players list.
- TopPanel now includes a currentWildColorLabel that becomes visible only when a wild card is active.

Reason:
- ArrayList is a simple way to mirror the order of players in the model and update the

scoreboard in O(n) time, where n is the number of players.
- The extra wild-colour label and event data reflect the new UNO Flip rules and AI wild-colour choices.

Summary

Compared to the Milestone 2 UML, the Milestone 3 design keeps the same core collections but extends them to support UNO Flip and AI behaviour:
- Card and the enums gained light/dark awareness and a Map for scoring.
- PileSystem still uses two Stack piles but now uses the isLightSide flags to build the deck.
- Player's hand remains an ArrayList, while ComputerPlayer extends Player to reuse the same data structure for AI.
- UNOModel's ArrayList and List are now paired with a UNOEvent class so that the model can push complete state snapshots to the view.
These changes are minimal but necessary to support the new game rules while keeping the data structures
simple, efficient, and easy to reason about.